NATURAL LANGUAGE PROCESSING

Sentiment Analysis of IMDb Movie Reviews

Assignment 1

Cover detail	Information
Course	Introduction to Natural Language Processing
Ahmad Iftikhar	L1F23BSCS1128
Aiza Shafiq	L1F23BSCS1135
Section	G-9

Dataset: IMDb 50K Movie Reviews

1. Topic and Dataset

This assignment focuses on preparing movie-review text for sentiment analysis. Sentiment analysis is a natural language processing task in which a system identifies the opinion or polarity expressed in text. In this dataset, each review is labelled as positive or negative.

1.1 Why this topic was selected

Movie reviews are familiar opinion-based text. They contain useful language phenomena for an introductory NLP task, including informal wording, punctuation, contractions, HTML remnants, and words whose meaning depends on context. The two sentiment labels also make the dataset suitable for a future supervised sentiment-classification task.

1.2 Dataset description

The project uses the IMDb Dataset of 50K Movie Reviews. The CSV contains 50,000 records and two columns: review and sentiment. The labels are balanced: 25,000 positive reviews and 25,000 negative reviews. The loading check found no missing values in either column.

Property	Value
Dataset size	50,000 reviews
Columns	review, sentiment
Positive labels	25,000 (50%)
Negative labels	25,000 (50%)
Missing values	0 in both columns

1.3 Dataset source and licence note

The dataset was obtained from the Kaggle page titled “IMDb Dataset of 50K Movie Reviews” by Lakshmi25npathi. It is associated with the original IMDb Large Movie Review Dataset research by Andrew L. Maas and colleagues (2011). The Kaggle page identifies the licence as “Other (specified in description)”; this report therefore does not claim unrestricted reuse rights. Users should review the dataset page and its terms before redistributing the data.

Kaggle dataset: https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews
Original research: https://ai.stanford.edu/~amaas/data/sentiment/

1.4 Project target

The target variable is the sentiment label (positive or negative). This assignment concentrates on understanding, tokenizing, and cleaning the review text so it can be used in a later machine-learning classification stage. No classifier accuracy is reported because model training and evaluation are outside the completed preprocessing work.

2. Text Exploration

The dataset was loaded with Python and pandas. Initial inspection included the dataset shape, column names, sample records, missing-value counts, and sentiment distribution.

- The dataset contains 50,000 rows and 2 columns.
- The review column contains the text; sentiment contains the class label.
- There are 25,000 positive and 25,000 negative reviews, so the classes are balanced.
- No missing values were reported in the review or sentiment columns.

2.1 Text characteristics and irregularities

The reviews include natural writing variations that need attention before analysis. The exploration and cleaning work considered HTML-like markup, URLs, inconsistent capitalization, punctuation, contractions, and repeated whitespace. Such patterns can create unnecessary token variations if they are not handled consistently.

The cleaning process preserves negation words and selected punctuation because they can affect the meaning or tone of a review. Examples include contractions such as “didn't” and emphasis marks such as “!” and “?”.

3. Tokenization and Regular Expressions

Tokenization divides a review into smaller units, usually words or punctuation tokens. The exploration notebook compares tokenization approaches using NLTK and regular expressions. This comparison shows how different rules treat contractions, punctuation, and non-word characters.

3.1 Regex-based tokenization

Regular expressions provide explicit rules for identifying token patterns. They are useful when a project needs control over which characters are retained and how text is split. The notebook demonstrates regex patterns and examines tokenization challenges rather than assuming that one pattern handles every case perfectly.

3.2 Challenges

- Contractions may be split differently depending on the tokenizer and pattern.
- Punctuation can carry emphasis or meaning, especially exclamation and question marks.
- HTML fragments and URLs may be treated as tokens unless removed or handled separately.
- A regex that is too broad can remove useful characters; one that is too permissive can retain noise.

4. Text Cleaning Pipeline

A reusable clean_text(text) function was implemented and applied to all 50,000 reviews. The pipeline follows a fixed sequence so that the same transformations can be repeated consistently on new review text.

Unicode normalization (NFKC): Standardizes compatible Unicode forms.

Lowercasing: Converts text to lowercase to reduce capitalization-only differences.

HTML and URL removal: Removes HTML tags and web links that are not useful for this task.

Special-character cleanup: Removes unwanted characters while retaining word characters, whitespace, apostrophes, and ! or ? punctuation.

Whitespace normalization: Collapses repeated spaces and trims extra whitespace.

Stopword removal: Removes common English stopwords while excluding negation terms from the removal list.

Lemmatization: Uses WordNet lemmatization with the verb part of speech to reduce inflected verb forms.

4.1 Preserving negation

Negation words can reverse the meaning of a review. For example, “not good” does not express the same sentiment as “good”. The stopword list was adjusted to preserve terms such as not, no, nor, don't, didn't, doesn't, isn't, wasn't, weren't, won't, wouldn't, couldn't, shouldn't, can't, and cannot.

4.2 Stemming and lemmatization

Porter stemming and WordNet lemmatization were examined. Stemming can produce shortened forms that are not always dictionary words (for example, “studies” may become “studi”). The implemented pipeline uses WordNet lemmatization with the verb part of speech. This choice is documented in the cleaning function.

4.3 Example

Input review	Cleaned output
This Movie was REALLY good! I didn't like the ending.	movie really good! didn't like end

5. Cleaning Results

The cleaning function was applied to the complete dataset. The generated CSV contains two columns, cleaned_review and sentiment, and retains all 50,000 rows.

Measure	Result
Original dataset shape	50,000 rows × 2 columns
Cleaned dataset shape	50,000 rows × 2 columns
Vocabulary before cleaning	390,931 unique whitespace-separated tokens
Vocabulary after cleaning	131,728 unique whitespace-separated tokens
Vocabulary reduction	259,203 tokens (approximately 66.3%)

Vocabulary counts were calculated by splitting text on whitespace and counting unique tokens. The values therefore reflect this counting method. The reduction indicates that normalization and removal of noise and common words reduced the number of distinct token forms.

6. Reproducibility

The project is maintained in a GitHub repository. It includes notebooks, Python scripts, a requirements file, README documentation, and Git ignore rules. The raw dataset is not committed because it is a large data file; each user should obtain it from the documented source and place it in the local data folder.

Repository: https://github.com/iahmad-1128/nlp-project-imdb-sentiment

Expected local input: data/IMDB Dataset.csv
Cleaned output: data/cleaned_imdb_dataset.csv

```
python scripts/load_data.py
python scripts/clean_data.py
```

The notebooks in the notebooks folder can also be opened and run in order: first the exploration/tokenization notebook, then the text-cleaning notebook. Install dependencies using the repository requirements file. The final submission should link to the exact Git commit tagged assignment-1 after all required files have been finalized.

7. Contribution Statement

The assignment was completed collaboratively by two members. Each member worked on a separate part of the preprocessing workflow, and the work was integrated in the shared GitHub repository.

Ahmad Iftikhar — Exploration and Tokenization

[Ahmad Iftikhar L1F23BSCS1128] was responsible for initial dataset exploration and tokenization. This included loading the IMDb dataset in the exploration notebook, checking the dataset structure and sentiment distribution, reviewing sample and irregular text, comparing tokenization approaches using NLTK and regular expressions, documenting regex patterns, and recording tokenization challenges.

The member committed the notebook work on a separate Git branch and submitted it for review and merging into the main branch.

Aiza Shafiq — Cleaning Pipeline and Documentation

[Aiza Shafiq L1F23BSCS1135] was responsible for the text-cleaning notebook and reusable Python scripts. This included implementing Unicode normalization, lowercasing, HTML/URL and special-character cleanup, whitespace normalization, stopword removal while preserving negation, and WordNet lemmatization. The member applied the cleaning function to the full dataset, measured vocabulary reduction, created the data-loading and cleaning scripts, and prepared README documentation. This work was committed on separate Git branches and integrated through pull requests.

Both members are responsible for reviewing the final repository, checking that the report matches the implemented work, and ensuring that the submitted files can be reproduced.

8. References and Acknowledgements

- Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning Word Vectors for Sentiment Analysis. Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies.
- Lakshmi25npathi. IMDb Dataset of 50K Movie Reviews. Kaggle. https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews
- NLTK documentation: https://www.nltk.org/
- Pandas documentation: https://pandas.pydata.org/docs/
- WordNet: https://wordnet.princeton.edu/

Acknowledgement: AI assistance was used for learning support, drafting guidance, and debugging during development. The team reviewed and adapted the work and remains responsible for the code, results, references, and final submission. Add any specific external tutorials or code sources actually used by the team before submission.